

SC2001 Project 1

Integration of Merge Sort & Insertion Sort

Handwritten Python algorithms

Measured inputs from 1,000 to 10,000,000 integers

The merge sort algorithm

Split first, then merge

```
def merge_sort(values, buffer=None):
    buffer = _get_buffer(values, buffer)
    lo = 0
    hi = len(values)
    _original(values, buffer, lo, hi)
```

```
def _original(values, buffer, lo, hi):
    size = hi - lo
    if size <= 1:
        return
    mid = lo + size // 2
    _original(values, buffer, lo, mid)
    _original(values, buffer, mid, hi)
    _merge(values, buffer, lo, mid, hi)
```

Merge function

```
def _merge(values, buffer, lo, mid, hi):
    left = lo
    right = mid
    output = lo
    while left < mid and right < hi:
        if values[left] <= values[right]:
            buffer[output] = values[left]
            left = left + 1
        else:
            buffer[output] = values[right]
            right = right + 1
        output = output + 1
    while left < mid:
        buffer[output] = values[left]
        left = left + 1
        output = output + 1
    while right < hi:
        buffer[output] = values[right]
        right = right + 1
        output = output + 1
    for output in range(lo, hi):
        values[output] = buffer[output]
```

The merge sort algorithm (with key comparisons)

Split first, then merge

```
def merge_sort_counted(values, buffer=None):
    buffer = _get_buffer(values, buffer)
    lo = 0
    hi = len(values)
    comparisons = _original_counted(values, buffer, lo, hi)
    return comparisons
```

```
def _original_counted(values, buffer, lo, hi):
    size = hi - lo
    if size <= 1:
        return 0
    mid = lo + size // 2
    left_comparisons = _original_counted(
        values, buffer, lo, mid)
    right_comparisons = _original_counted(
        values, buffer, mid, hi)
    merge_comparisons = _merge_counted(
        values, buffer, lo, mid, hi)
    comparisons = (left_comparisons +
        right_comparisons + merge_comparisons)
    return comparisons
```

Merge function

```
def _merge_counted(values, buffer, lo, mid, hi):
    comparisons = 0
    left = lo
    right = mid
    output = lo
    while left < mid and right < hi:
        comparisons = comparisons + 1
        if values[left] <= values[right]:
            buffer[output] = values[left]
            left = left + 1
        else:
            buffer[output] = values[right]
            right = right + 1
        output = output + 1
    while left < mid:
        buffer[output] = values[left]
        left = left + 1
        output = output + 1
    while right < hi:
        buffer[output] = values[right]
        right = right + 1
        output = output + 1
    for output in range(lo, hi):
        values[output] = buffer[output]
    return comparisons
```

The insertion sort algorithm

Check ranges first, ensure it is valid, before performing the sort

```
def insertion_sort(values, lo=0, hi=None):  
    if hi is None:  
        hi = len(values)  
    _check_range(values, lo, hi)  
    _insert(values, lo, hi)
```

Insertion sort function

```
def _insert(values, lo, hi):  
    for position in range(lo + 1, hi):  
        key = values[position]  
        previous = position - 1  
        while previous >= lo:  
            if values[previous] > key:  
                values[previous + 1] = values[previous]  
                previous = previous - 1  
            else:  
                break  
        values[previous + 1] = key
```

The insertion sort algorithm (with key comparisons)

Check ranges first, ensure it is valid, before performing the sort

```
def insertion_sort_counted(values, lo=0, hi=None):
    if hi is None:
        hi = len(values)
    _check_range(values, lo, hi)
    comparisons = _insert_counted(values, lo, hi)
    return comparisons
```

Insertion sort function

```
def _insert_counted(values, lo, hi):
    comparisons = 0
    for position in range(lo + 1, hi):
        key = values[position]
        previous = position - 1
        while previous >= lo:
            comparisons = comparisons + 1
            if values[previous] > key:
                values[previous + 1] = values[previous]
                previous = previous - 1
            else:
                break
        values[previous + 1] = key
    return comparisons
```

The hybrid algorithm

Stop splitting when the range has at most **S** items

```
def hybrid_sort(values, threshold, buffer=None):  
    _check_threshold(threshold)  
    buffer = _get_buffer(values, buffer)  
    lo = 0  
    hi = len(values)  
    _hybrid(values, buffer, lo, hi, threshold)
```

```
def _hybrid(values, buffer, lo, hi, threshold):  
    size = hi - lo  
    if size <= threshold:  
        _insert(values, lo, hi)  
        return  
    mid = lo + size // 2  
    _hybrid(values, buffer, lo, mid, threshold)  
    _hybrid(values, buffer, mid, hi, threshold)  
    _merge(values, buffer, lo, mid, hi)
```

Worked example: S=3

Input: [5, 2, 4, 1, 3, 6]

Leaves: [5, 2, 4] and [1, 3, 6]

Insertion: [2, 4, 5] and [1, 3, 6]

Merge: [1, 2, 3, 4, 5, 6]

The hybrid algorithm (with key comparisons)

Stop splitting when the range has at most **S** items

```
def hybrid_sort_counted(values, threshold, buffer=None):  
    _check_threshold(threshold)  
    buffer = _get_buffer(values, buffer)  
    lo = 0  
    hi = len(values)  
    comparisons = _hybrid_counted(  
        values, buffer, lo, hi, threshold)  
    return comparisons
```

```
def _hybrid_counted(values, buffer, lo, hi, threshold):  
    size = hi - lo  
    if size <= threshold:  
        comparisons = _insert_counted(values, lo, hi)  
        return comparisons  
    mid = lo + size // 2  
    left_comparisons = _hybrid_counted(  
        values, buffer, lo, mid, threshold)  
    right_comparisons = _hybrid_counted(  
        values, buffer, mid, hi, threshold)  
    merge_comparisons = _merge_counted(  
        values, buffer, lo, mid, hi)  
    comparisons = (left_comparisons +  
        right_comparisons + merge_comparisons)  
    return comparisons
```

Worked example: **S=3**

Input: [5, 2, 4, 1, 3, 6]

Leaves: [5, 2, 4] and [1, 3, 6]

Insertion: [2, 4, 5] and [1, 3, 6]

Merge: [1, 2, 3, 4, 5, 6]

$3 + 2 + 5 = 10$ key comparisons

Correctness, counting and fair timing

Correctness and counting

Insertion maintains a sorted prefix.
Merge emits the smallest remaining head.
Induction gives a sorted, stable result.

Count each evaluated data comparison:
merge $\leq$ and insertion $>$.
Include terminating false comparisons.

Experiment protocol

Uniform integers in $[1, 1,000,000]$.
Three seeds per case, paired inputs.
Separate counted and uncounted runs.
CPU clock: `time.process_time()`.

Exclude RNG, copies, buffer allocation,
validation and I/O. Run sequentially.
Verify order, length and exact histogram.

Input Data - Random Dataset Generation

Generation of array in increasing size from 1,000 to 10 million in the range of [1, x]

For this experiment, we set x to 1000000 (1 million). So [1, 1000000].

Random Dataset

```
array = [random.randint(1, 100) for _ in range(1000)]
```

Run 1: [51, 82, 23, 41, ..., 3]

Run 2: [44, 14, 66, 59, ..., 7]

WHY FIXED SEED?

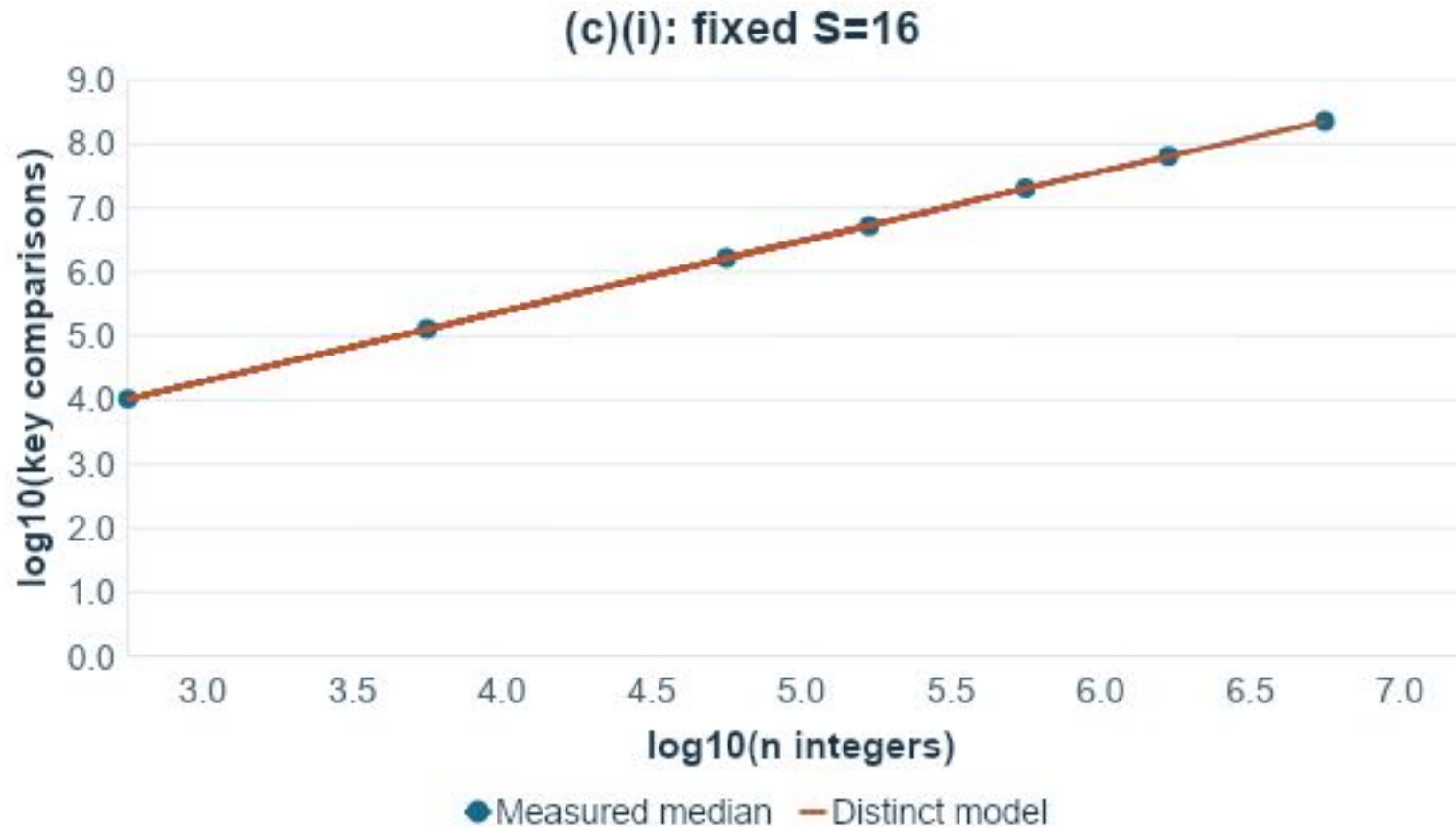
Same code != same data

Different data can affect performance

Makes comparison of different S values unreliable

SOLUTION: random.seed(seed)

Fixed S retains $n \log n$ growth



Original

$T(n) = 2T(n/2) + \Theta(n)$

Time: $\Theta(n \log n)$

Buffer: $\Theta(n)$

Hybrid

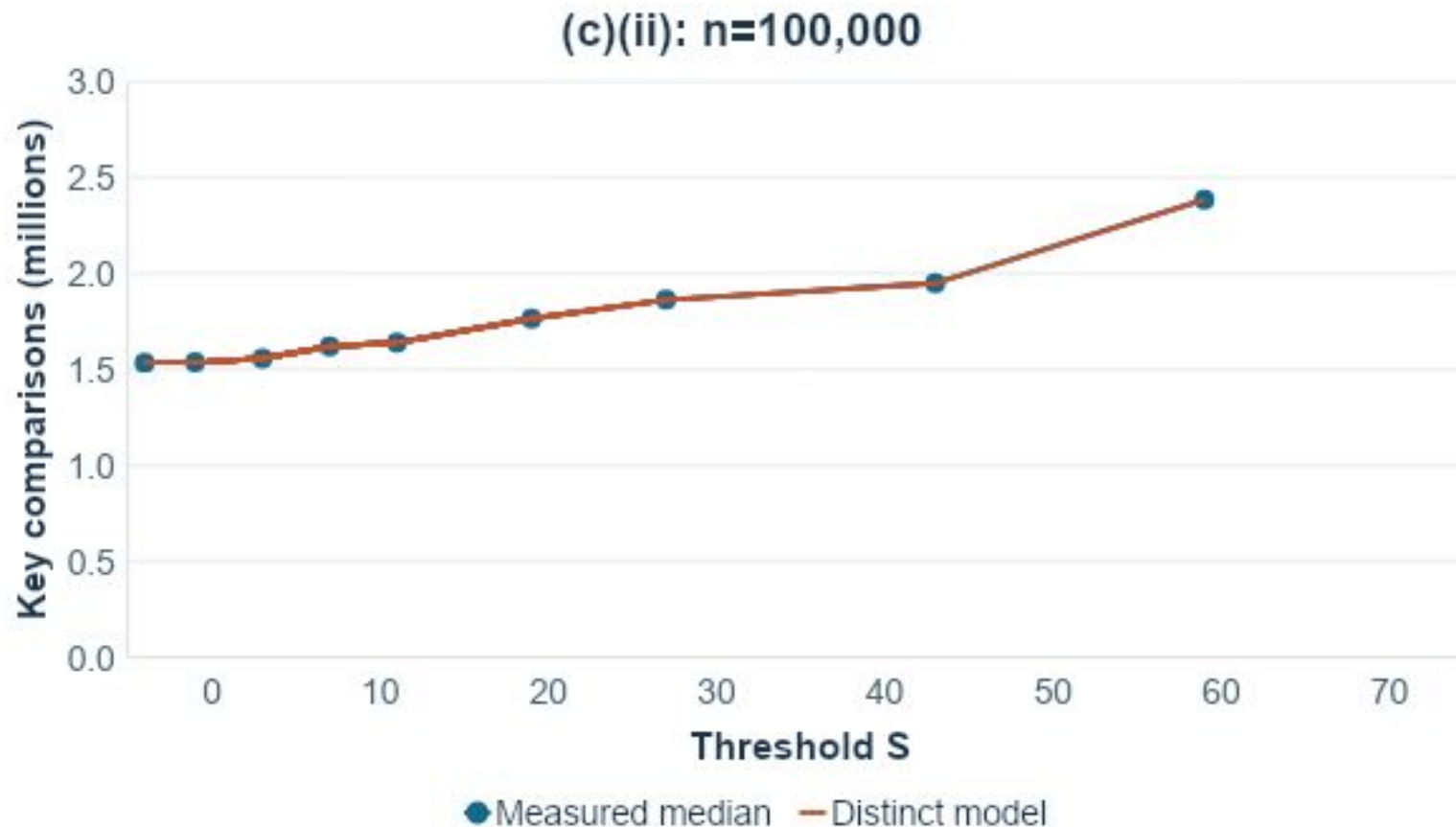
About $\log_2(n/S)$ merge levels

Random leaf work: $\Theta(nS)$

Fixed S : $\Theta(n \log n)$

Both axes show $\log_{10}$ values. Analytical counts use actual rounded splits with no fitted scale factor.

Increasing S changes the leaf work



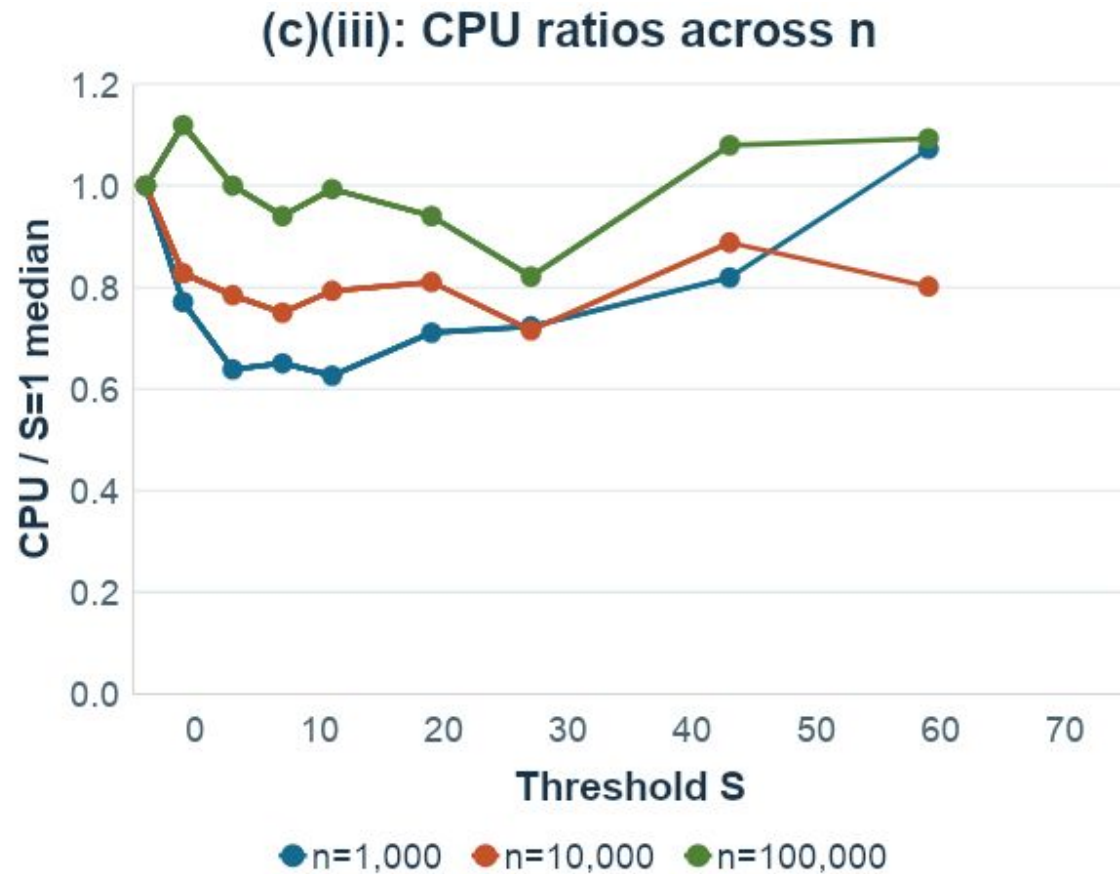
Fewer merge levels
Larger insertion leaves

Random insertion: $\Theta(S^2)$
per leaf
About n/S leaves
Total leaf work: $\Theta(nS)$

Some S values share the
same leaves.
Plateaus are expected.

$S=1$ is included. Medians of three seeds. Full plots also show ranges and the worst-case bound.

CPU tuning selected S=12 by the tie-break rule

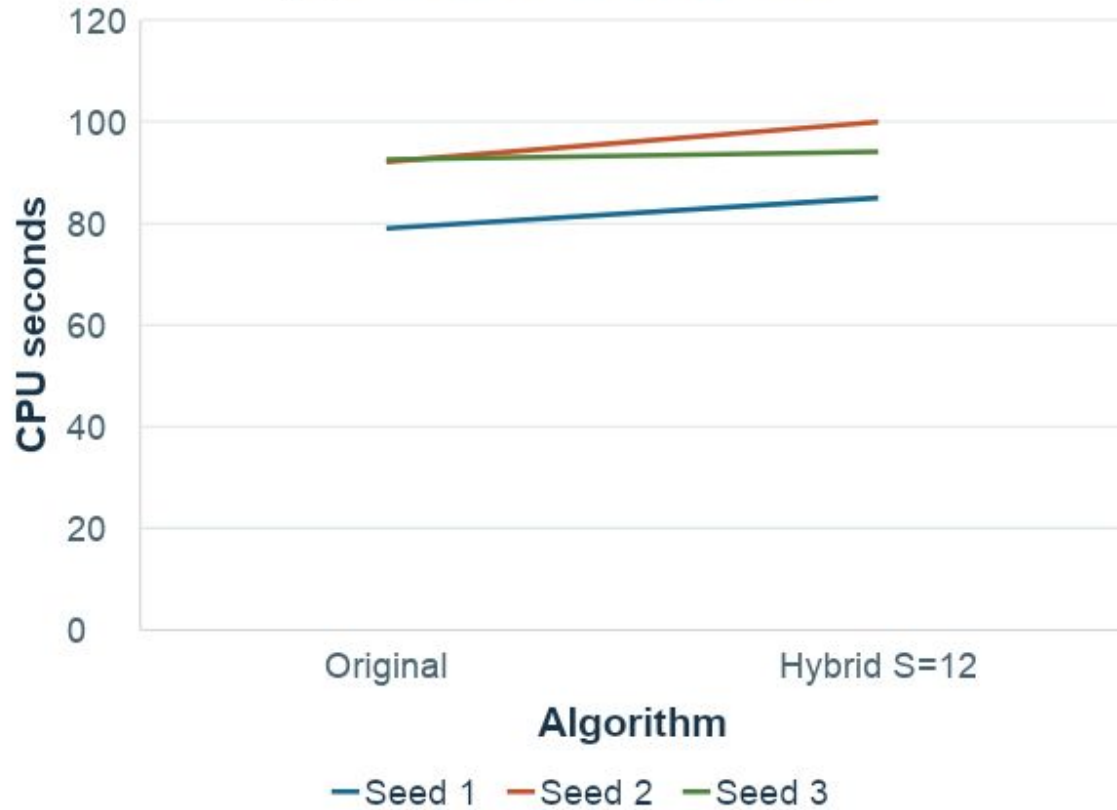


S=12, 32 tied at 2.78125 s median. The rule chooses smaller S. Three seeds per case.

Original and hybrid on 10 million integers

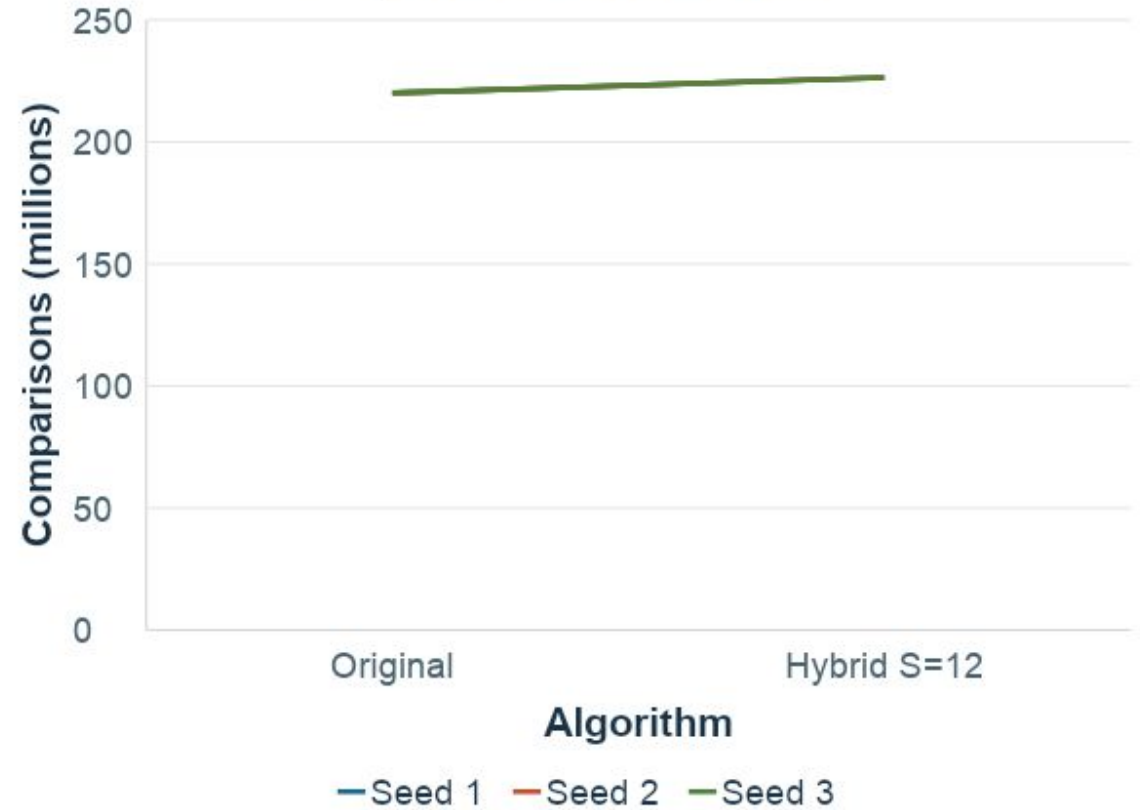
Evaluated on 3 independent random seeds ($n \sim 10000000$, $x = 1000000$), optimal $S = 12$ from Part (c)

(d): uncounted CPU time



Original median: 92.09 s
Hybrid median: 94.08 s

Separate counted runs



Original: ~220.1 M comparisons
Hybrid: ~226.4 M comparisons

Speedup: 0.929x

Why Hybrid Lost in Python?

High Python Loop Overhead: Insertion sort's inner while loop suffers from heavy interpreter overhead (dynamic type checks, bounds checking, pointer reassignment per shift).

Recursion Savings Nullified: Merge sort recursion is $O(\log n)$ function calls. Saving the bottom 3–4 recursion levels saves very little time compared to the cost of millions of interpreted inner-loop shifts.

Comparison Cost: In Python, executing an extra 6.32M > evaluations incurs direct CPU execution penalties.

Conclusions and questions

Hybrid was slower in 3 of 3 final CPU pairs.
Median paired speedup: 0.929x.

CPU time and key comparisons measure different costs.

S=12 follows a finite, noisy search and an explicit tie-break.

One interpreter, one machine, uniform integer inputs.

2 minutes of Q&A

Appendix: exact comparison models

Merge sort

Worst comparisons:

$$n \lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1$$

Random distinct merge of lengths a,b:

$$a+b - a/(b+1) - b/(a+1)$$

Insertion expectation for m items:

$$m(m-1)/4 + m - H_m$$

$$H_m = 1 + 1/2 + \dots + 1/m$$

Hybrid regimes

Worst/average (random distinct):

$$\Theta(n \log(n/S) + nS), \text{ for } S \leq n$$

Fixed S: $\Theta(n \log n)$

$S = \log n$: $\Theta(n \log n)$

$S = n^\alpha$: $\Theta(n^{1+\alpha})$

$S \geq n$: insertion only

Buffer $O(n)$, recursion $O(\log n)$

Appendix: raw final measurements

Seed	Algorithm	S	CPU seconds	Key comparisons
100900100	original	1	78.968750	220,095,927
100900100	hybrid	12	85.000000	226,418,149
100900101	original	1	92.093750	220,099,689
100900101	hybrid	12	99.906250	226,424,963
100900102	original	1	92.515625	220,101,452
100900102	hybrid	12	94.078125	226,417,959

Each CPU value is from an uncounted run. Counts come from a separate run on identical unsorted data.

Appendix: different criteria, different winners

Input n	Lowest median CPU S	Fewest median comparisons S
1,000	16	1
10,000	32	1
100,000	32	1

Ties use the smaller S. These are per-size descriptive winners, before shortlist confirmation.